

# THE DATA VIZ TOOLKIT

## *Part 1: the options panel behind your plots*

Everything Modules 8 and 9 let you draw, and the knobs they told you to go find yourself

|                               |                                          |
|-------------------------------|------------------------------------------|
| The canvas & polish           | tight_layout, savefig, and friends       |
| Making bars talk              | labels, lines, and the third bar plot    |
| Taming skew                   | log scales, boxen, and honest histograms |
| Figure-level mastery          | legends, spines, themes, color           |
| Scatter & distribution extras | regression lines, bubbles, raw points    |
| Pandas plotting extras        | the fast lane, expanded                  |
| The cheat sheet               | the whole toolkit on one page            |

### HEADS UP

Every code block here was run on the real stack before it went in (seaborn 0.13.2, matplotlib 3.11, pandas 3.0). If a line does not work for you, check your versions first, a couple of these are newer than your notebook thinks.

*Master of Data Science · CMPINF-2100 · Modules 8-9 Companion · Tools, not concepts*

# Table of Contents

| # | Section                                  | The one-line reason to read it                       |
|---|------------------------------------------|------------------------------------------------------|
| 0 | <b>How to Read This</b>                  | the premise, and the taught-vs-new tags              |
| 1 | <b>The Canvas &amp; Polish</b>           | stop your labels from colliding, save a clean figure |
| 2 | <b>Making Bars Talk</b>                  | put numbers on bars, and meet <code>barplot</code>   |
| 3 | <b>Taming Skew</b>                       | the tools for the data that broke your boxplot       |
| 4 | <b>Seaborn Figure-Level Mastery</b>      | legends, spines, themes, and color you control       |
| 5 | <b>Scatter &amp; Distribution Extras</b> | the regression line they promised you                |
| 6 | <b>Pandas Plotting Extras</b>            | the quick-look tool, with more knobs                 |
| 7 | <b>The Cheat Sheet</b>                   | if you reread one page, this one                     |

Every tool follows the same rhythm: **the idea** in plain words, **the syntax** you can paste into a cell, **when and why** you would reach for it (and what it replaces), and **the trap** if it has one. The traps are where the points live.

## 0. How to Read This

Modules 8 and 9 taught you the **plot picker**: which chart goes with which data (bar for one category, histogram for one number, scatter for two numbers, box/violin/point for a category against a number). That was the map. This Trainer is the **options panel** behind each of those charts, the arguments and helper functions that turn a rough plot into a finished one.

The course kept punting those arguments to self-study: "look it up in the Seaborn docs," "check the properties tutorial," or just use `.help`. Fair, but slow. So this is the curated tour, so you know *what* to look up. The lookup habit itself, once: in Jupyter type `sns.histplot?` for the quick signature, or `help(sns.histplot)` for the full text.

### Two tags you will see on every tool

**NEW** the course never showed this. This is the meat of the Trainer.

**DEEPER** you were taught this, but there is more to it than you saw. Included so the new stuff has somewhere to attach.

One promise, same as every Trainer: every code sample here runs. It was executed on the stack above before printing. Lines starting with `#` are notes to you, not code that does anything.

# 1. The Canvas & Polish

Everything seaborn draws sits on a matplotlib **figure** (the whole picture) made of one or more **axes** (the individual plots). Module 8 taught you to build that scaffold with `fig, ax = plt.subplots()` and size it with `figsize=(w, h)`. This section is the finishing work: the calls that fix the ugly parts and get a figure out of the notebook and into a report.

## 1.1 `tight_layout()` and `layout="constrained"` NEW

**The idea.** Multi-panel figures collide with themselves: titles overlap the panel above, y-labels run off the edge, tick labels crash into each other. Two calls fix it automatically instead of by hand.

```
# one-shot fix, call it right before you show/save
fig, axes = plt.subplots(2, 2, figsize=(7, 5))
# ... draw into the four axes ...
plt.tight_layout()          # nudges spacing so nothing overlaps (default pad=1.08)
plt.show()

# the modern, continuous version: let a layout engine manage it
fig, axes = plt.subplots(2, 2, figsize=(7, 5), layout="constrained")
```

**When / why.** Any time a multi-subplot figure looks cramped or clipped. `tight_layout()` is the one-shot; `layout="constrained"` (or `"tight"`) hands the whole figure to a layout engine that re-solves the spacing continuously and handles legends and colorbars more gracefully. Reach for `constrained` when you are hand-building matplotlib figures; seaborn's figure-level functions (`relplot`, `displot`, `catplot`) already manage their own layout, so you rarely need it there.

### The Trap: `layout=` is newer than you think

The `layout="constrained"` string needs matplotlib 3.6+. On an older install you write `plt.subplots(..., constrained_layout=True)` instead. If `layout=` throws, that is why.

## 1.2 `fig.suptitle()` and `subplots_adjust()` NEW

**The idea.** `ax.set_title()` (taught) titles *one* panel. `fig.suptitle()` sets a single title over the *whole* figure, which is what you want above a grid of subplots. `subplots_adjust()` is the manual version of `tight_layout` when you want exact spacing.

```
fig, axes = plt.subplots(1, 2, figsize=(7, 3))
axes[0].set_title("Adelie")          # per-panel title
axes[1].set_title("Gentoo")
fig.suptitle("Flipper length by species")  # one title over everything
fig.subplots_adjust(top=0.85, wspace=0.3)  # manual spacing: fractions of the figure
```

**When / why.** `suptitle` for a headline over a facet grid. `subplots_adjust` when `tight_layout`'s automatic choice is not quite what you want (for example, to reserve room at the top for that `suptitle`).

### 1.3 `savefig()`, getting the figure out NEW

**The idea.** A plot in a notebook is not a deliverable. `savefig` writes it to a file for a report, a slide, or a submission.

```
fig, ax = plt.subplots()
sns.countplot(data=penguins, x="species", ax=ax)
fig.savefig("species.png", dpi=150, bbox_inches="tight", transparent=True)
```

**When / why.** `dpi=150` (or 300) sets resolution so it is not blurry when printed.

`bbox_inches="tight"` trims the whitespace border. `transparent=True` drops the white background so it sits on any slide color.

#### The Trap: in Jupyter, save in the same cell

The notebook's inline backend **closes the figure once the cell finishes rendering**. If you draw in one cell and call `savefig` in the next, you save an empty canvas. Put the `savefig` in the same cell as the plot, before it displays. (In a plain `.py` script this does not happen, so the advice is Jupyter-specific.)

### 1.4 `plt.close()`, the loop cleanup NEW

**The idea.** Every `plt.subplots()` or seaborn figure-level call opens a figure that stays in memory until closed. In a loop that adds up fast.

```
num_cols = gap.select_dtypes("number").columns
for col in num_cols:
    fig = sns.displot(data=gap, x=col, kind="hist").figure
    fig.savefig(f"{col}.png", dpi=120, bbox_inches="tight")
    plt.close(fig)          # free it now; do not pile up open figures
```

**When / why.** Exactly the "loop `displot` over every numeric column" pattern Module 8 showed.

Without `plt.close`, matplotlib warns after 20 open figures and your memory climbs.

`plt.close("all")` clears every open figure at once.

## 2. Making Bars Talk

You learned the count bar chart (`countplot`) and the mean-with-error point plot (`pointplot`). Bars have a few more moves the course skipped.

### 2.1 `ax.bar_label()`, put the numbers on the bars NEW

**The idea.** A reader should not have to trace a bar down to the axis to read its value. Print the value right on the bar.

```
fig, ax = plt.subplots()
sns.countplot(data=penguins, x="species", ax=ax)
ax.bar_label(ax.containers[0])      # -> 152, 68, 124 sit above the bars
```

**When / why.** Any bar chart an audience will read a number off of. `fmt="%.1f"` formats the labels (one decimal), `padding=3` lifts them off the bar, `label_type="center"` puts them inside. For grouped (hue) bars, loop: `for c in ax.containers: ax.bar_label(c)`.

### The Trap: it needs the container, and a modern matplotlib

`bar_label` labels a *bar container*, not the axes, hence `ax.containers[0]`. And it only exists on matplotlib 3.4+. On an older install it is simply missing.

## 2.2 A line over the bars: `ax.plot()` and `axhline()` NEW

**The idea.** Sometimes you want a line across a bar chart: a trend traced over the bar tops, or a single reference line (a mean, a target, a threshold).

```
counts = penguins["species"].value_counts().sort_index()

fig, ax = plt.subplots()
ax.bar(counts.index, counts.values)
ax.plot(range(len(counts)), counts.values, marker="o")  # a trend line over the tops
ax.axhline(counts.mean(), color="crimson", linestyle="--")  # reference line at the mean
```

**When / why.** `ax.plot(...)` over the bar tops draws a combined bar+line (a trend across categories). `ax.axhline(y)` draws a horizontal reference line across the whole axes, and `ax.axvline(x)` a vertical one, the idiomatic way to mark "the average" or "the cutoff" on any plot, far cleaner than building the line by hand.

## 2.3 `sns.barplot()`, the third bar chart NEW

**The idea.** There are three different "bar" plots and they answer three different questions. This is the one the course never named.

```
fig, ax = plt.subplots()
# each bar = the MEAN of total_bill per day, with a 95% CI
sns.barplot(data=tips, x="day", y="total_bill", ax=ax)
```

| Function               | What each bar/point is                           | You were       |
|------------------------|--------------------------------------------------|----------------|
| <code>countplot</code> | a <b>count</b> of rows in that category          | taught (Mod 8) |
| <code>pointplot</code> | a <b>mean</b> of a number, drawn as a point + CI | taught (Mod 9) |
| <code>barplot</code>   | a <b>mean</b> of a number, drawn as a bar + CI   | <b>new</b>     |

**When / why.** `barplot` is `pointplot` with bars instead of dots: same mean, same default 95% CI error bar. Use it when bars read better than points for your audience. Change the summary with `estimator="median"`, and drop the error bar with `errorbar=None`.

### The Trap: `barplot` is not `countplot`

A `barplot` with no `y` does not "count", it needs a numeric `y` to average. If you want counts, that is `countplot`. Mixing them up is the classic first-week bar-chart error. (Version note: the `estimator="mean" / errorbar=("ci", 95)` spelling is seaborn 0.12+, which your course notes already use, so you are fine.)

## 2.4 `order=`, and rotating labels NEW DEEPER

**The idea.** Seaborn draws categories alphabetically or in first-seen order. `order=` (and `hue_order=`) forces a meaningful sequence. And long category names overlap unless you rotate them.

```
ideal = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
fig, ax = plt.subplots()
sns.countplot(data=diamonds, x="cut", order=ideal, ax=ax)  # NEW: your order, not
                                                           alphabetical
ax.tick_params(axis="x", labelrotation=45)                # taught back in Module 8
# plt.xticks(rotation=45) is the NEW shortcut when you have no ax
```

**When / why.** `order=` whenever the categories have a natural sequence (sizes, grades, days) that alphabetical order destroys. The axes-level `ax.tick_params(labelrotation=45)` was taught in the pandas plotting note; the genuinely new sibling is `plt.xticks(rotation=45)`, the pyplot-state one-liner for when you did not keep an `ax` variable.

## 3. Taming Skew

Module 8 was honest that **boxplots fall apart on skewed data** (diamond price, GDP per capita) and pointed you to faceted histograms and KDE. Those work. Here are three more tools for the same problem, and one that finally makes the skew readable in place.

### 3.1 Log-scale an axis NEW

**The idea.** When most values are small and a few are huge, a linear axis crushes the small ones into a sliver. A log scale spreads them back out.

```
fig, ax = plt.subplots()
sns.histplot(data=diamonds, x="price", ax=ax)
ax.set_yscale("log")  # matplotlib way: log the y (or x) axis of any plot

# seaborn shortcut, for distribution plots only:
sns.displot(data=diamonds, x="price", log_scale=True)  # logs the x axis for you
```

**When / why.** Any heavily right-skewed variable. `ax.set_yscale("log") / set_xscale("log")` works on *any* matplotlib axes; `log_scale=True` is the seaborn convenience baked into `histplot/displot`. It *replaces* squinting at a plot where 90% of the data is jammed against the left edge.

### 3.2 `sns.boxenplot()`, the boxplot that survives skew NEW

**The idea.** A boxplot draws one box (the middle 50%) and dumps everything in the tails as individual outlier dots. On skewed data that is hundreds of dots and no shape. A **boxen plot** (letter-value plot) draws many nested boxes instead, so the tail becomes readable structure.

```
fig, ax = plt.subplots(figsize=(7, 3))
sns.boxenplot(data=diamonds, x="color", y="price", ax=ax)
```

**When / why.** Large, skewed data across categories. For diamond price by color, a boxplot buries each color's upper tail under a cloud of outlier dots (hundreds per color, roughly 3,100 across the whole plot); the boxen plot draws about **15 to 19 nested boxes** per color instead, so the skew shows as narrowing steps rather than a smear of dots. It is a "better boxplot," not a replacement for the faceted-histogram / KDE advice from Module 8, another tool for the same job.

#### The Trap: the argument name changed

On seaborn 0.13 the box-width control is `width_method=`. On 0.12 it was `scale=`. Your course proves you are on 0.12+, but not necessarily 0.13, so if a `boxenplot` keyword errors, this is the likely culprit.

### 3.3 Histogram `stat=` and `binwidth=` NEW DEEPER

**The idea.** You learned `bins=` (how many bars). Two more knobs matter. `stat=` changes what the y-axis *measures*, and `binwidth=` sets bar width in real units instead of a count.

```
sns.histplot(data=penguins, x="flipper_length_mm", stat="probability") # bars sum to 1
sns.histplot(data=penguins, x="flipper_length_mm", binwidth=5)        # 5 mm-wide bins
```

| stat=             | Y-axis is       | The bars/area total              |
|-------------------|-----------------|----------------------------------|
| "count" (default) | rows per bin    | heights sum to n (342)           |
| "probability"     | share of rows   | heights sum to 1.0               |
| "percent"         | percent of rows | heights sum to 100               |
| "density"         | density         | <b>area</b> = 1 (heights do not) |

**When / why.** `stat="probability"` or `"percent"` lets you compare two groups of *different sizes* on the same y-axis (raw counts would just show that one group is bigger). `binwidth=5` is more honest than `bins=20` when the unit means something ("5 mm bins") rather than an arbitrary bar count.

## 4. Seaborn Figure-Level Mastery

The figure-level functions (`relplot`, `displot`, `catplot`) gave you `hue/col/row` and `height/aspect`. Here is the rest of the control panel: layout, legends, spines, whole-look themes, and color.

### 4.1 `col_wrap=` DEEPER

**The idea.** You saw this once in Module 8. It is worth a second look: `col_wrap=N` wraps a long row of `col=` facets into an N-wide grid so they do not run off the page.

```
sns.displot(data=penguins, x="flipper_length_mm", col="species", col_wrap=2)
```

**When / why.** Any facet variable with more than three or four levels. Without it, ten facets draw as one unreadable ten-panel row.

### 4.2 `despine()` and `move_legend()` NEW

**The idea.** Two quick cleanups. `despine()` removes the top and right frame lines (they carry no information). `move_legend()` relocates a legend that is sitting on top of your data.

```
ax = sns.scatterplot(data=penguins, x="flipper_length_mm", y="bill_depth_mm", hue="species")
sns.despine(ax=ax)                                # drop the top + right border lines
sns.move_legend(ax, "upper left", bbox_to_anchor=(1, 1)) # legend outside, top-right
```

**When / why.** `despine` for a cleaner, less boxed-in look (add `offset=` / `trim=` for more). `move_legend` replaces the fight with raw `ax.legend(loc=...)` placement; `bbox_to_anchor=(1, 1)` parks it just outside the top-right corner so it never covers a point.

### 4.3 `set_theme()`, change everything at once NEW

**The idea.** One call sets the global look of every plot after it: the background style, the size scaling, and the default color palette.

```
sns.set_theme(style="whitegrid", context="talk", palette="deep")
# style    = background: darkgrid | whitegrid | dark | white | ticks
# context = sizing:      paper | notebook | talk | poster (bigger text for slides)
# call sns.set_theme() with no args to snap back to the default (darkgrid, notebook)
```

**When / why.** Set it once at the top of a notebook. `context="talk"` or `"poster"` bumps every font and line so a plot is legible on a projector. The granular versions are `set_style`, `set_context`, `set_palette`.



### Note: darkgrid vs whitegrid is the background, not the grid

Both styles show a grid. The difference is the panel color: **darkgrid** is a light-gray panel with white gridlines, **whitegrid** is a white panel with gray gridlines. Pick by what your page or slide background is.

## 4.4 Labeling a figure-level plot after the fact **NEW**

**The idea.** Figure-level calls return a `FacetGrid` object with its own methods for labels and titles, because the plot spans many axes.

```
g = sns.relplot(data=penguins, x="flipper_length_mm", y="bill_depth_mm", col="species")
g.set_axis_labels("Flipper length (mm)", "Bill depth (mm)")  # all panels at once
g.set_titles("{col_name}")                                  # template per panel title
g.figure.suptitle("Penguins by species")
```

**When / why.** When `ax.set_xlabel(...)` does not reach a multi-panel seaborn figure. These methods label every facet in one call.

## 4.5 palette= and color=, control your colors **NEW**

**The idea.** The course never once chose a color. `palette=` colors the hue groups; `color=` sets one color for a plot with no hue.

```
sns.countplot(data=penguins, x="species", hue="species", palette="Set2")
sns.scatterplot(data=penguins, x="flipper_length_mm", y="bill_depth_mm",
                hue="species", palette="viridis")
sns.histplot(data=penguins, x="flipper_length_mm", color="teal")  # one color, no hue
```

**When / why.** To distinguish groups clearly or match a report's colors. A palette is a named set ("Set2", "viridis", "rocket") or a list of colors.

### Accessibility, for the reports real people read

Prefer perceptually-even, colorblind-safe palettes: "viridis" or "colorblind". Avoid a red/green split as the only signal; roughly 8% of men cannot separate them. Never let color be the *only* thing carrying meaning.

# 5. Scatter & Distribution Extras

The scatter and distribution plots from Module 9 have the richest set of extras, including the one tool the lecture explicitly promised was "coming later."

## 5.1 regplot / lmpplot, the regression line you were promised **NEW**

**The idea.** The Module 9 scatter lecture said a best-fit line was coming. This is it: a scatter *with* a fitted regression line and a confidence band.

```
# axis-level (one plot):
sns.regplot(data=penguins, x="flipper_length_mm", y="body_mass_g")

# figure-level (adds hue / col / row, like relplot): one line PER group
sns.lmplot(data=penguins, x="flipper_length_mm", y="body_mass_g", hue="species")
```

**When / why.** To show the direction and strength of a linear relationship, not just the point cloud. `lmplot` is to `regplot` what `relplot` is to `scatterplot`: figure-level, so it takes `hue/col/row` and fits a separate line per group. `order=2` fits a curve (polynomial); `ci=None` drops the band; `lowess=True` fits a wiggly local trend instead of a straight line.

### The payoff: this is Simpson's paradox, drawn

Module 9 warned that a trend can reverse within groups. `lmplot(..., hue="species")` draws a separate line per group; set it beside the un-hued fit (which draws the single pooled line) and you can see whether the within-group slopes disagree with the overall trend.

## 5.2 Bubble and shape: `size=` and `style=` NEW

**The idea.** You mapped a category to color with `hue`. You can map more variables to point *size* and point *shape*, stacking as many as five variables onto one scatter (`x`, `y`, `color`, `size`, `shape`).

```
sns.scatterplot(data=penguins, x="flipper_length_mm", y="bill_depth_mm",
                hue="species",                        # color = species
                size="body_mass_g",                   # point SIZE = body mass (a bubble chart)
                sizes=(20, 200),                     # min/max point area
                style="sex")                          # marker SHAPE = sex
```

### The Trap: `size=` is not `s=`

Lowercase `s=` (taught, e.g. `s=250`) sets one **fixed** size for every point. `size="body_mass_g"` **maps a variable** to size, so each point differs. Same three letters, opposite jobs.

## 5.3 Show the raw points: `stripplot` / `swarmplot` NEW

**The idea.** A box or violin summarizes; sometimes you want to see the actual observations too. Overlay them on the same axes.

```
fig, ax = plt.subplots()
sns.boxplot(data=penguins, x="species", y="flipper_length_mm", ax=ax)
sns.stripplot(data=penguins, x="species", y="flipper_length_mm",
              color="black", size=2, ax=ax)  # jittered raw points on top
```

**When / why.** To show sample size and gaps a summary hides. `stripplot` jitters points sideways so they do not stack; `swarmplot` packs them so none overlap (better for small n, slow for large n). Overlay on a box/violin for "summary plus the truth behind it."

## 5.4 Histogram overlap and shape: `multiple=`, `element=`, `cumulative=` NEW

**The idea.** When a histogram has a hue, the groups overlap and hide each other. These arguments decide how they share the space.

```
# multiple="stack": groups stacked on top of each other
sns.histplot(data=penguins, x="flipper_length_mm", hue="species", multiple="stack")
# multiple="fill": 100% stacked, compare the group mix
sns.histplot(data=penguins, x="flipper_length_mm", hue="species", multiple="fill")
# element="step": clean outlines instead of bars
sns.histplot(data=penguins, x="flipper_length_mm", hue="species", element="step")
# cumulative=True: draw a CDF
sns.histplot(data=penguins, x="flipper_length_mm", cumulative=True)
```

**When / why.** `multiple="stack"` or `"dodge"` to stop groups occluding each other; `"fill"` to compare the group *mix* across the x-axis (each slice sums to 100%); `element="step"` for clean outlines when several groups overlap; `cumulative=True` to read "what fraction is below X."

## 5.5 `jointplot`, a scatter with margins NEW

**The idea.** One call draws the scatter *and* a histogram of each variable along its axis, so you see the relationship and both distributions at once.

```
sns.jointplot(data=penguins, x="flipper_length_mm", y="bill_depth_mm", kind="scatter")
# kind= scatter | hist | hex | kde | reg (reg adds a regression line + margins)
```

**When / why.** When the marginal distributions matter as much as the relationship. `kind="hex"` is the density fix for an overplotted scatter (the honeycomb version of the 2D histogram you already know).

### Where Part 2 picks up

`jointplot` handles **one** x-y pair with its margins. The matrix that does this for *every* pair of variables at once, the **pair plot**, opens Module 10 and is the first tool in Part 2 of this Trainer.

# 6. Pandas Plotting Extras

Module 8 taught the quick lane: `series.plot.bar()` and `series.plot.hist()` call matplotlib for you, no seaborn needed. A few more knobs make it genuinely useful.

## 6.1 More one-liners: `.plot.box()`, `.plot.kde()`, `.plot.area()` NEW

```
penguins["flipper_length_mm"].plot.box()    # quick boxplot, no seaborn
penguins["flipper_length_mm"].plot.kde()    # quick density curve
```

**When / why.** A fast look at one column's shape without building a seaborn call. Same quick-and-dirty spirit as the bar/hist you were taught.

(Note: `.plot.kde()` needs SciPy, which ships with Anaconda and the course environment; a bare pandas install raises `ImportError`.)

## 6.2 `subplots=`, `secondary_y=`, `logy=` NEW DEEPER

```
num = penguins[["flipper_length_mm", "body_mass_g", "bill_length_mm"]]
num.plot(subplots=True, layout=(1, 3), figsize=(9, 3))    # each column in its own panel

# two series on very different scales, on one plot:
weather = pd.DataFrame({"temp_C": [20, 22, 19, 24], "sales": [1200, 1500, 900, 2100]})
weather.plot(secondary_y=["sales"], title="Two scales")    # sales on a right-hand y-axis
```

**When / why.** `subplots=True` with `layout=(r, c)` is the fastest way to eyeball several columns at once. `secondary_y=[...]` puts chosen columns on a right-hand axis so a big and a small series share one plot. `logy=True/logx=True` log an axis inline. (`title=`, `rot=`, `figsize=` you already knew.)

### The rule stays the same

Pandas plotting is the quick-look tool (it just calls matplotlib under the hood). Reach for seaborn or matplotlib directly the moment you want control or polish. These knobs stretch the quick lane; they do not replace the destination.

## 7. The Cheat Sheet

The whole toolkit, compressed. If you reread one page before you next open a notebook, this one.

| Tool                                                                                     | One-line job                                                   | New?   |
|------------------------------------------------------------------------------------------|----------------------------------------------------------------|--------|
| <code>tight_layout()</code> / <code>layout="constrained"</code>                          | stop panels/labels from overlapping                            | new    |
| <code>fig.suptitle()</code>                                                              | one title over a whole grid                                    | new    |
| <code>savefig(dpi=, bbox_inches="tight")</code>                                          | write a clean figure to a file (save in-cell in Jupyter)       | new    |
| <code>plt.close(fig)</code>                                                              | free figures in a loop                                         | new    |
| <code>ax.bar_label(ax.containers[0])</code>                                              | print values on bars (mpl 3.4+)                                | new    |
| <code>ax.axhline()</code> / <code>axvline()</code>                                       | a reference line (mean, target)                                | new    |
| <code>sns.barplot</code>                                                                 | bars of a <b>mean</b> + CI (not counts)                        | new    |
| <code>order=</code> / <code>hue_order=</code>                                            | force a meaningful category order                              | new    |
| <code>ax.set_yscale("log")</code> / <code>log_scale=True</code>                          | spread out skewed data                                         | new    |
| <code>sns.boxenplot</code>                                                               | boxplot that survives skew (0.13: <code>width_method=</code> ) | new    |
| <code>stat="probability"</code> / <code>binwidth=</code>                                 | compare unequal groups; real-unit bins                         | new    |
| <code>despine()</code> / <code>move_legend()</code>                                      | drop the frame; move the legend out                            | new    |
| <code>set_theme(style=, context=)</code>                                                 | set the global look once                                       | new    |
| <code>palette=</code> / <code>color=</code>                                              | control colors (use colorblind-safe)                           | new    |
| <code>regplot</code> / <code>lmplot</code>                                               | scatter + regression line (per group)                          | new    |
| <code>size=</code> / <code>style=</code>                                                 | map variables to bubble size / marker shape                    | new    |
| <code>stripplot</code> / <code>swarmplot</code>                                          | raw points over a box/violin                                   | new    |
| <code>multiple=</code> / <code>element=</code> / <code>cumulative=</code>                | stack/fill/step histograms; a CDF                              | new    |
| <code>jointplot</code>                                                                   | scatter + marginal distributions                               | new    |
| <code>.plot.box/ .kde/ .area</code> , <code>subplots=</code> , <code>secondary_y=</code> | more pandas quick-look knobs                                   | new    |
| <code>col_wrap=</code> , <code>tick_params(labelrotation=)</code>                        | wrap facets; rotate labels                                     | deeper |

### The five habits that finish a plot

1. Multi-panel figure looks cramped? `plt.tight_layout()` or `layout="constrained"`.
2. Data jammed in a corner? Log the axis (`set_yscale("log") / log_scale=True`).
3. Bars an audience will read? `ax.bar_label()`.
4. Legend covering the data? `sns.move_legend(ax, "upper left", bbox_to_anchor=(1,1))`.
5. Saving for a report? `savefig(dpi=150, bbox_inches="tight")`, in the same cell.

That is the options panel behind every Module 8-9 plot. None of it changes *which* chart you pick, that was the last two modules. All of it changes how finished the chart looks when you are done. Part 2 picks up at the pair plot, where Module 10 begins.

---

*Authored and directed by **DatJavaClass (Victor S)**, who conceived, structured, formatted, fact-checked, and edited these notes, with assistance by Claude. Some material may have been derived from assigned material, but has not been copied verbatim. For source materials please contact CMPINF-2100 Faculty and Assistants.*